DESIGN AND ANALYSIS OF ALGORITHMS

EXPERIMENT REPORT

Experiment No.: 39

Analyze Binary Search Performance for Datasets with Frequent Updates Requiring Re-sorting

Student Name	T . KODANDA RAM
Register Number	192524162
Department	AI & DS
Course	Design and Analysis of Algorithms (DAA)
Programming Language	Python
Faculty Name	Dr. F. Mary Harin Fernandez
Date	06/08/2026

2. Abstract

The objective of this experiment is to analyze the performance of the Binary Search algorithm when it is applied to a dataset that undergoes frequent updates in the form of insertion and deletion operations, and to measure the additional cost introduced by the re-sorting that these updates make necessary. The implementation reads a dataset of integers, sorts it with the built-in Python list sort, and performs a Binary Search while recording the execution time using time.perf_counter(). An element is then inserted into the dataset using the list append operation, which destroys the sorted order of the data, so the dataset is re-sorted and searched again. Subsequently an element is deleted using the list remove operation, the dataset is re-sorted once more, and a third Binary Search is executed. Each of these eight operations is individually timed and the values are displayed in a consolidated performance comparison table, along with the complexity of every operation used and a short result analysis. The expected result is that Binary Search itself remains extremely fast because it runs in logarithmic time, while the repeated re-sorting required after each update dominates the total cost of maintaining the dataset. The experiment therefore demonstrates the practical trade-off between the maintenance cost of keeping a dataset sorted and the search efficiency that a sorted dataset provides.

3. Aim

To implement Binary Search in Python on a user-supplied dataset, to perform insertion and deletion operations that require the dataset to be re-sorted, to measure the execution time of every sorting, searching, insertion and deletion operation, and to analyze the trade-off between the maintenance cost of re-sorting and the search efficiency obtained from Binary Search.

4. Objectives

1. To read a dataset of integers from the user and display it in its original unsorted form.
2. To sort the dataset using the built-in Python sort operation and measure the initial sorting time.
3. To implement the iterative Binary Search algorithm and locate a user-specified key in the sorted dataset.
4. To perform an insertion by append and a deletion by remove, recording the time of each update.
5. To re-sort the dataset after each update and measure the re-sorting time separately.
6. To tabulate all recorded execution times and analyze the trade-off between maintenance cost and search efficiency.

5. Problem Statement

Binary Search can be applied only to a dataset maintained in sorted order, yet in practice datasets are not static: elements are inserted and deleted frequently. Here an insertion appends the new element at the end of the list and a deletion removes an element from an arbitrary position. Both leave the dataset

in a state where sorted order is no longer guaranteed, so it must be re-sorted before the next Binary Search can be performed correctly.

The problem, as stated by the program itself, is to analyze Binary Search performance for datasets with frequent updates requiring re-sorting, to measure the update and search execution times, and to analyze the trade-off between maintenance cost and search efficiency. The task therefore requires the initial sorting, initial search, insertion, re-sorting after insertion, search after insertion, deletion, resorting after deletion and search after deletion to be timed individually and compared. Maintenance cost is the time spent updating and re-sorting, while search efficiency is the time consumed by the searches, and the comparison of the two decides whether keeping the dataset sorted is justified for a given usage pattern.

6. Introduction

Binary Search locates a key by repeatedly halving the region in which the key can lie. Here the search begins with left at the first index and right at the last index. Each iteration computes mid as (left + right) // 2 and returns mid if the middle element equals the key; if the middle element is smaller, left moves to mid + 1, and if it is greater, right moves to mid - 1. The loop runs while left is less than or equal to right, and returns minus one if it ends without a match.

The correctness of this procedure depends entirely on the dataset being in ascending sorted order, since discarding one half is valid only because every element on one side of the middle is smaller and every element on the other side larger. On unsorted data the algorithm may discard the half that contains the key, so the program sorts the dataset immediately after reading it and before the first search.

Updates disturb this ordering. The insertion appends the new element at the end of the list, so a small value added to a list of larger values breaks the ascending order at once. The deletion removes an element from an arbitrary position, and the program re-sorts after it as well so that the dataset is guaranteed valid before the next search. Re-sorting is thus the mechanism that restores the precondition of Binary Search after every update.

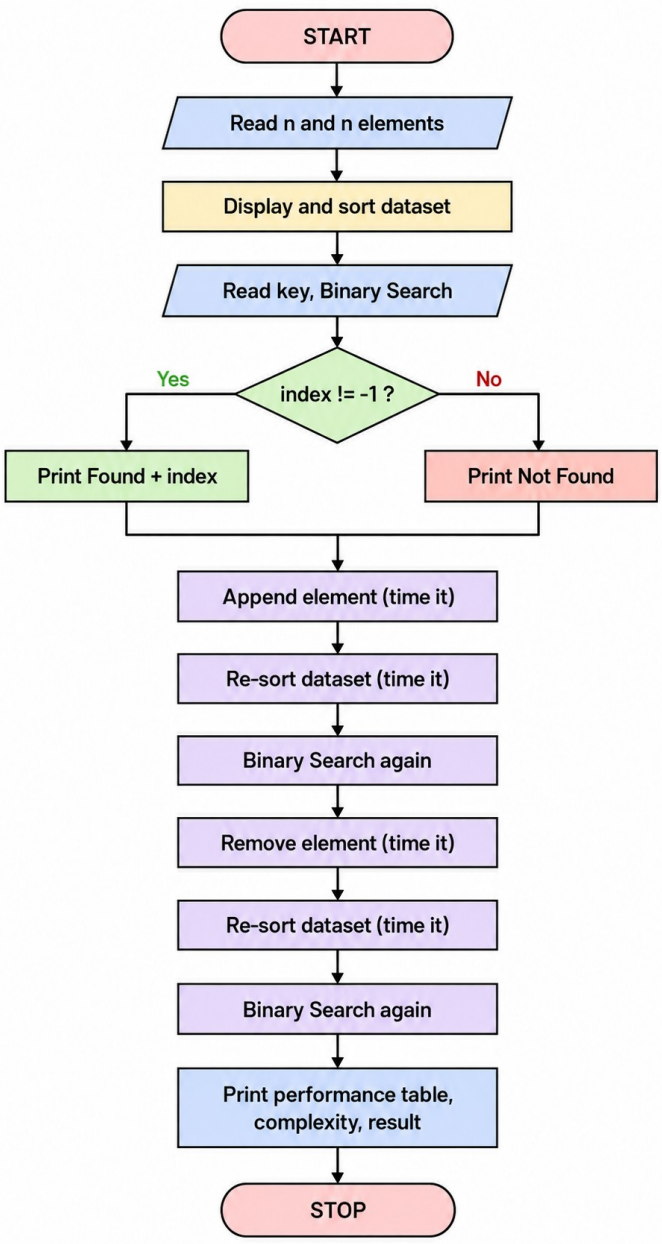
Execution time is measured because theoretical complexity alone does not show how the operations compare in practice. The program uses the high-resolution timer time.perf_counter() around each operation and prints the elapsed time with eight decimal places, giving direct evidence of how much time is spent searching and how much on keeping the dataset sorted.

7. Algorithm

1. Start.
2. Read the number of elements n and read n integers into the list data.
3. Display the initial unsorted dataset.
4. Time the in-place sort of the dataset and display the sorted dataset with the sorting time.

5. Read the search key, time a Binary Search on the dataset, and print the index if found or a not-found message, with the search time.
6. In Binary Search set left to 0 and right to the last index; while left <= right compute mid = (left + right) // 2; if data[mid] equals the key return mid; if data[mid] < key set left = mid + 1; else set right = mid - 1; if the loop ends return -1.
7. Read the element to insert, time its append to the dataset, then re-sort, timing the re-sort, and display the dataset with both timings.
8. Perform Binary Search again and record the search time after insertion.
9. Read the element to delete, time its removal if present with a suitable message, then re-sort, timing the re-sort, and display the dataset with both timings.
10. Perform Binary Search again and record the search time after deletion.
11. Display the performance comparison table with all eight execution times, followed by the complexity analysis and the result analysis.
12. Stop.

8. Flowcha



9. Python Program

```
import time
# ------------------------------------
# DISPLAY DATASET
# ------------------------------------
def display_dataset(data):
    print("Dataset :", data)
# ------------------------------------
# SORT DATASET
# ------------------------------------
def sort_dataset(data):
    start = time.perf_counter()
    data.sort()
    end = time.perf_counter()
    return end - start
# ------------------------------------
# BINARY SEARCH
# ------------------------------------
def binary_search(data, key):
    left = 0
    right = len(data) - 1
    while left <= right:
        mid = (left + right) // 2
        if data[mid] == key:
            return mid
        elif data[mid] < key:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

```
# ------------------------------------------------
# SEARCH WITH EXECUTION TIME
# ------------------------------------------------
def search_element(data):
    key = int(input("\nEnter element to search : "))
    start = time.perf_counter()
    index = binary_search(data, key)
    end = time.perf_counter()
    search_time = end - start
    if index != -1:
        print("Result : Element Found at Index", index)
    else:
        print("Result : Element Not Found")
    print("Search Time : {:.8f} seconds".format(search_time))
    return search_time
# ------------------------------------------------
# INSERT ELEMENT
# ------------------------------------------------
def insert_element(data):
    element = int(input("\nEnter element to insert : "))
    start = time.perf_counter()
    data.append(element)
    end = time.perf_counter()
    insert_time = end - start
    return insert_time
```

```
# ------------------------------------------------
# DELETE ELEMENT
# ------------------------------------------------
def delete_element(data):
    element = int(input("\nEnter element to delete : "))
    start = time.perf_counter()
    if element in data:
        data.remove(element)
        print("Element Deleted Successfully.")
    else:
        print("Element Not Present.")
    end = time.perf_counter()
    delete_time = end - start
    return delete_time
# ------------------------------------------------
# DISPLAY PERFORMANCE TABLE
# ------------------------------------------------
def performance_table(sort1,
                      search1,
                      insert,
                      resort1,
                      search2,
                      delete,
                      resort2,
                      search3):
    print("\n")
    print("=" * 70)
    print("PERFORMANCE COMPARISON")
    print("=" * 70)
    print("{:<30} {:>20}".format("Operation", "Execution Time (s)"))
    print("-" * 70)
    print("{:<30} {:>20.8f}".format("Initial Sorting", sort1))
    print("{:<30} {:>20.8f}".format("Initial Search", search1))
    print("{:<30} {:>20.8f}".format("Insertion", insert))
    print("{:<30} {:>20.8f}".format("Re-sorting After Insert", resort1))
    print("{:<30} {:>20.8f}".format("Search After Insert", search2))
    print("{:<30} {:>20.8f}".format("Deletion", delete))
    print("{:<30} {:>20.8f}".format("Re-sorting After Delete", resort2))
    print("{:<30} {:>20.8f}".format("Search After Delete", search3))
    print("=" * 70)
# ------------------------------------------------
# RESULT ANALYSIS
# ------------------------------------------------
def result_analysis():
    print("\n")
    print("=" * 70)
    print("RESULT ANALYSIS")
    print("=" * 70)
    print("• Binary Search provides fast searching on sorted datasets.")
    print("• Frequent insertions and deletions require re-sorting.")
    print("• Re-sorting increases maintenance cost.")
    print("• Binary Search is suitable when searches are frequent")
    print("  and updates are comparatively less frequent.")
    print("=" * 70)
```

```
    # ------------------------------------------------
# MAIN PROGRAM
# ------------------------------------------------
def main():
    print("=" * 70)
    print(" BINARY SEARCH PERFORMANCE WITH FREQUENT UPDATES ")
    print("=" * 70)
    print("\nProblem Statement")
    print("-" * 70)
    print("Analyze Binary Search performance for datasets with")
    print("frequent updates requiring re-sorting.")
    print("Measure update and search execution times and")
    print("analyze the trade-off between maintenance cost")
    print("and search efficiency.")
    # ---------------- INPUT ---------------- #
    n = int(input("\nEnter number of elements : "))
    data = []
    print("Enter the elements:")
    for i in range(n):
        value = int(input())
        data.append(value)
    # ---------------- INITIAL SORT ---------------- #
    print("\nInitial Dataset")
    display_dataset(data)
    sort_time = sort_dataset(data)
    print("\nSorted Dataset")
    display_dataset(data)
    print("\nInitial Sorting Time : {:.8f} seconds".format(sort_time))
```

```
# ---------------- INITIAL SEARCH ---------------- #
print("\n")
print("=" * 70)
print("INITIAL BINARY SEARCH")
print("=" * 70)
search_time1 = search_element(data)
# ---------------- INSERT ---------------- #
print("\n")
print("=" * 70)
print("INSERT OPERATION")
print("=" * 70)
insert_time = insert_element(data)
resort_time1 = sort_dataset(data)
print("\nDataset After Insertion")
display_dataset(data)
print("Insertion Time            : {:.8f} seconds".format(insert_time))
print("Re-sorting Time           : {:.8f} seconds".format(resort_time1))
print("\nBinary Search After Insertion")
search_time2 = search_element(data)
# ---------------- DELETE ---------------- #
print("\n")
print("=" * 70)
print("DELETE OPERATION")
print("=" * 70)
delete_time = delete_element(data)
resort_time2 = sort_dataset(data)
print("\nDataset After Deletion")
display_dataset(data)
print("Deletion Time             : {:.8f} seconds".format(delete_time))
print("Re-sorting Time           : {:.8f} seconds".format(resort_time2))
print("\nBinary Search After Deletion")
search_time3 = search_element(data)
```

```
# ---------------- PERFORMANCE TABLE ----------------
performance_table(
    sort_time,
    search_time1,
    insert_time,
    resort_time1,
    search_time2,
    delete_time,
    resort_time2,
    search_time3
)
# ---------------- COMPLEXITY ---------------- #
print("\n")
print("=" * 70)
print("COMPLEXITY ANALYSIS")
print("=" * 70)
print("Sorting (Python Sort)    : O(n log n)")
print("Binary Search            : O(log n)")
print("Insertion (List Append)  : O(1)")
print("Deletion                 : O(n)")
print("Re-sorting After Update  : O(n log n)")
# ---------------- RESULT ANALYSIS ---------------- #
result_analysis()
```

```
    # ---------------- CONCLUSION ---------------- #
    print("\n")
    print("=" * 70)
    print("CONCLUSION")
    print("=" * 70)
    print("Binary Search performs very efficiently on sorted")
    print("datasets with logarithmic search time.")
    print("However, frequent insertions and deletions require")
    print("re-sorting, increasing the maintenance cost.")
    print("Hence, Binary Search is most suitable for")
    print("applications where searching is frequent and")
    print("updates are relatively infrequent.")
    print("\nProgram Executed Successfully.")
# ------------------------------------------------
# DRIVER CODE
# ------------------------------------------------
if __name__ == "__main__":
    main()
```

10. Program Explanation

display_dataset() — Purpose: to print the current contents of the dataset. Input: the list data. Output: a printed line showing the list elements. Working: it prints the list with one print statement and is called before sorting, after sorting, after insertion and after deletion.

sort_dataset() — Purpose: to sort the dataset in ascending order and measure the sorting time. Input: the list data. Output: the elapsed sorting time in seconds. Working: it times the in-place list sort with perf_counter and returns the elapsed time. It is invoked three times: once initially and once after each update.

binary_search() — Purpose: to locate a key in the sorted dataset. Input: the sorted list data and the integer key. Output: the index of the key, or minus one if absent. Working: left starts at 0 and right at the last index; each iteration computes mid, returns mid on a match, sets left = mid + 1 when the middle element is smaller and right = mid - 1 otherwise, returning minus one when left exceeds right. **search_element()** — Purpose: to read the key, invoke Binary Search, time it and report the outcome. Input: the sorted list data and the key entered by the user. Output: the search time in seconds; it also prints the result. Working: it times the call to binary_search(), prints the index if found or a not-found message otherwise, and prints the search time to eight decimal places.

insert_element() — Purpose: to insert a new element and measure the insertion time. Input: the list data and the element entered by the user. Output: the insertion time in seconds. Working: it times the append of the element at the end of the list. Because the element is appended, the sorted order is disturbed and re-sorting becomes necessary.

delete_element() — Purpose: to delete an element and measure the deletion time. Input: the list data and the element entered by the user. Output: the deletion time in seconds; it also prints whether deletion succeeded. Working: it times a check for the element followed by its removal, printing a success message if present and a not-present message otherwise.

performance_table() — Purpose: to display all recorded execution times in one table. Input: the eight measured times for the sorts, searches, insertion and deletion. Output: a formatted table of operations against execution times. Working: it prints a heading, a column header, and one width-aligned row per measured value with eight decimal places.

result_analysis() — Purpose: to print the concluding observations of the run. Input: no parameters. Output: four printed observation statements. Working: it prints that Binary Search is fast on sorted data, that updates require re-sorting, that re-sorting raises maintenance cost, and that Binary Search suits frequent searches with infrequent updates.

main() — Purpose: to control the flow by invoking the other functions in order. Input: the dataset size and elements, the search keys, and the elements to insert and delete. Output: the complete printed output of the experiment. Working: it prints the title and problem statement, reads and displays the dataset, sorts it, performs the initial search, then the insertion with re-sorting and a second search, then the deletion with re-sorting and a third search, and finally calls performance_table() and result_analysis().

(Insert Sample Input Here)

11. Sample Output

```
================================================
 BINARY SEARCH PERFORMANCE WITH FREQUENT UPDATES
================================================

Problem Statement
------------------------------------------------
Analyze Binary Search performance for datasets with
frequent updates requiring re-sorting.
Measure update and search execution times and
analyze the trade-off between maintenance cost
and search efficiency.

Enter number of elements : 5
Enter the elements:
5
3
8
10
1

Initial Dataset
Dataset : [5, 3, 8, 10, 1]

Sorted Dataset
Dataset : [1, 3, 5, 8, 10]

Initial Sorting Time : 0.00000264 seconds


================================================
INITIAL BINARY SEARCH
================================================

Enter element to search : 9
Result : Element Not Found
Search Time : 0.00000571 seconds
```

```
================================================
INSERT OPERATION
================================================

Enter element to insert : 9

Dataset After Insertion
Dataset : [1, 3, 5, 8, 9, 10]
Insertion Time            : 0.00000116 seconds
Re-sorting Time           : 0.00000198 seconds

Binary Search After Insertion

Enter element to search : 9
Result : Element Found at Index 4
Search Time : 0.00000383 seconds


================================================
DELETE OPERATION
================================================

Enter element to delete : 5
Element Deleted Successfully.

Dataset After Deletion
Dataset : [1, 3, 8, 9, 10]
Deletion Time             : 0.00005588 seconds
Re-sorting Time           : 0.00000166 seconds

Binary Search After Deletion

Enter element to search : 10
Result : Element Found at Index 4
Search Time : 0.00000283 seconds
```

```
================================================
PERFORMANCE COMPARISON
================================================
Operation                        Execution Time (s)
------------------------------------------------
Initial Sorting                        0.00000264
Initial Search                         0.00000571
Insertion                              0.00000116
Re-sorting After Insert                0.00000198
Search After Insert                    0.00000383
Deletion                               0.00005588
Re-sorting After Delete                0.00000166
Search After Delete                    0.00000283
================================================



================================================
COMPLEXITY ANALYSIS
================================================
Sorting (Python Sort)    : O(n log n)
Binary Search            : O(log n)
Insertion (List Append)  : O(1)
Deletion                 : O(n)
Re-sorting After Update  : O(n log n)
```

```
============================================================
RESULT ANALYSIS
============================================================
• Binary Search provides fast searching on sorted datasets.
• Frequent insertions and deletions require re-sorting.
• Re-sorting increases maintenance cost.
• Binary Search is suitable when searches are frequent
  and updates are comparatively less frequent.
============================================================



============================================================
CONCLUSION
============================================================
Binary Search performs very efficiently on sorted
datasets with logarithmic search time.
However, frequent insertions and deletions require
re-sorting, increasing the maintenance cost.
Hence, Binary Search is most suitable for
applications where searching is frequent and
updates are relatively infrequent.

Program Executed Successfully.
```

12. Output Analysis

Initial sorting: the entered dataset [5, 3, 8, 10, 1] became [1, 3, 5, 8, 10] in 0.00000264 seconds, establishing the precondition of Binary Search. Binary Search: the search for 9 on this dataset returned minus one and printed that the element was not found, taking 0.00000571 seconds, the largest search timing because an unsuccessful search runs until the interval is empty.

Insertion: appending 9 took 0.00000116 seconds, the smallest update timing, since no element is shifted. Re-sorting: the dataset became [1, 3, 5, 8, 9, 10] in 0.00000198 seconds, more than the insertion itself, showing that maintaining order costs more than the update. Search after insertion: 9 was found at index 4 in 0.00000383 seconds, faster than the earlier unsuccessful search.

Deletion: removing 5 printed a success message and took 0.00005588 seconds, the largest measured value, because the list is scanned and later elements are shifted. Re-sorting: [1, 3, 8, 9, 10] was resorted in 0.00000166 seconds. Search after deletion: 10 was found at index 4 in 0.00000283 seconds, the fastest search. Overall the table shows the three searches remaining near a few microseconds while the deletion alone exceeded all searches and both re-sorts combined.

13. Time Complexity

Operation	Complexity
Sorting	O(n log n)
Binary Search	O(log n)
Insertion (Append)	O(1)
Deletion	O(n)
Re-sorting	O(n log n)

Sorting is O(n log n) because the built-in list sort used in sort_dataset() is comparison based and needs a logarithmic number of passes over all n elements. Binary Search is O(log n) because each iteration discards half of the remaining interval. Insertion by append is O(1) because the element is placed at the end and nothing is moved. Deletion is O(n) because remove scans the list and shifts every later element one position left. Re-sorting is O(n log n) because the whole dataset is sorted again after every update, twice in this program.

14. Space Complexity

The space complexity is O(n). The only data structure maintained is the single list data holding the n entered integers, which grows by one after the insertion and shrinks by one after the deletion. binary_search() uses only the variables left, right and mid, and the timing code uses only start and end, so these add constant space. Sorting is performed in place on the same list, so no second datasetsized structure is created. The memory requirement therefore grows linearly with the number of elements.

15. Performance Analysis

Search speed: the three searches took 0.00000571, 0.00000383 and 0.00000283 seconds and did not grow as the dataset changed. Update time: the insertion took 0.00000116 seconds while the deletion took 0.00005588 seconds, because appending is constant time whereas removal requires a scan and shifting. Re-sorting overhead: the two re-sorts took 0.00000198 and 0.00000166 seconds, greater than the insertion itself and growing as O(n log n). Maintenance cost: the update and re-sorting timings together exceed the three search timings, so most effort goes into keeping the dataset sorted. Search efficiency: the searches stay fast because each runs on a freshly sorted dataset, making this arrangement economical only when searches greatly outnumber updates.

16. Advantages

1. Binary Search locates an element in logarithmic time, each search finishing in a few microseconds.
2. The iterative binary_search() uses only three extra variables, so extra space is constant.
3. Every operation is timed separately with time.perf_counter(), giving precise per-stage costs.
4. Insertion by append is constant time, making the update itself very cheap.
5. Re-sorting after each update guarantees the precondition of Binary Search, keeping results correct.
6. The performance table shows all eight timings together, making the trade-off directly visible.

17. Limitations

1. Every update forces an O(n log n) re-sort, costlier than the search it enables.
2. Insertion appends at the end instead of the correct sorted position, so order is always destroyed.
3. The deletion uses remove, which is O(n) and was the slowest operation in the run.
4. Only one insertion and one deletion are handled per execution.
5. The dataset is small and entered manually, so timings vary with system conditions.

18. Applications

1. Searching a sorted database index where lookups outnumber modifications.
2. Looking up a word in a dictionary or a name in a telephone directory.

3. Finding a roll number in a sorted list of enrolled candidates.
4. Locating a product code in a sorted inventory listing of a store.
5. Searching a value in a sorted table of marks or rankings.
6. Retrieving an entry from a sorted list of library accession numbers.

19. Result

The program executed successfully and produced the expected output. The dataset [5, 3, 8, 10, 1] was sorted into [1, 3, 5, 8, 10], the search for 9 correctly reported not found, the insertion of 9 with resorting produced [1, 3, 5, 8, 9, 10] where 9 was found at index 4, and the deletion of 5 with re-sorting produced [1, 3, 8, 9, 10] where 10 was found at index 4. The implementation therefore demonstrates Binary Search on a sorted dataset, the effect of updates on sorted order, the necessity of re-sorting, and the measurement and tabulation of all eight execution times.

20. Conclusion

The experiment establishes that Binary Search performs very efficiently on a sorted dataset, completing each search in a few microseconds with no significant growth as the dataset changed, consistent with its logarithmic complexity. The updates disturbed this efficiency indirectly: the append based insertion placed the new element out of order, while removal required a linear scan followed by shifting, making deletion the most expensive operation of the run at 0.00005588 seconds. Since neither update preserved sorted order, re-sorting had to follow each of them to restore the precondition on which the correctness of Binary Search depends, and this re-sorting costs O(n log n), more than the O(log n) search it enables. The trade-off is therefore between update cost and search efficiency: keeping a dataset sorted is advantageous when searches are frequent and updates rare, whereas under very frequent updates the repeated re-sorting overhead can outweigh the gain from fast searching.

21. References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th Edition, MIT Press, 2022.

2. E. Horowitz, S. Sahni and S. Rajasekaran, Fundamentals of Computer Algorithms, 2nd Edition, Universities Press, 2008.

3. Python Software Foundation, The Python Standard Library Documentation, https://docs.python.org/3A

4. A. V. Aho, J. E. Hopcroft and J. D. Ullman, The Design and Analysis of Computer Algorithms, Pearson Education, 1974.

5. D. E. Knuth, The Art of Computer Programming, Volume 3: Sorting and Searching, 2nd Edition, Addison-Wesley, 1998.